

实验六：无人机避障

实验日期：2025.11.28

姓名：何宗霖

学号：23374275

班级：231516

任课教师：李家军

一、实验目的

1. 掌握在 FPGA 上驱动 16×16 LED 点阵的方法
2. 理解点阵显示的逐列扫描机制
3. 熟悉像素帧（frame）合成的设计方式
4. 通过 Verilog 实现一个简化无人机避障游戏
5. 使用 FPGA 进行交互式游戏逻辑开发
6. 理解按键消抖、状态机、节拍控制等重要设计方法

二、实验原理

1. 整体架构

- 主模块 `game_logic`：管理无人机、管道、碰撞检测、帧生成
- 显示模块 `led_driver`：逐列扫描 16×16 点阵
- 输入模块：按键/开关控制跳跃与重置
- 通过 `frame[0..15]`（每列 16bit）传递屏幕内容

2. 游戏逻辑设计

- 无人机（ 3×3 ）通过上下移动（重力 + 跳跃）实现控制
- Pipe（障碍物）在 `playfield` 中左移并周期性生成
- `Frame = bird` 显示层 OR `playfield` 层
- 检测 `bird` 与 `pipe` 的像素重叠作为碰撞

3. `playfield` 与无人机的独立存储

- `playfield`：16 列，每周期左移 1 列
- 无人机：3 列 $\times$ 16 行，随高度移动进行位移
- 最终显示：对 `bird` 列与 `playfield` 做 OR 合成

4. 显示驱动逻辑（`led_driver`）

- 逐列扫描方式：每个时钟输出下一列数据
- `col_sel` 控制哪一列点亮
- `row_data` 输出 16bit 行像素
- 使用 `frame_flat` 传输 256 bit 显存

5. Game Over 显示方案

- 在 `game_over` 状态下固定显示
- 显示 GOOD GAME 字样（ 16×16 ）
- 字模通过 `frame[i] = 16'bxxxx` 手动赋值

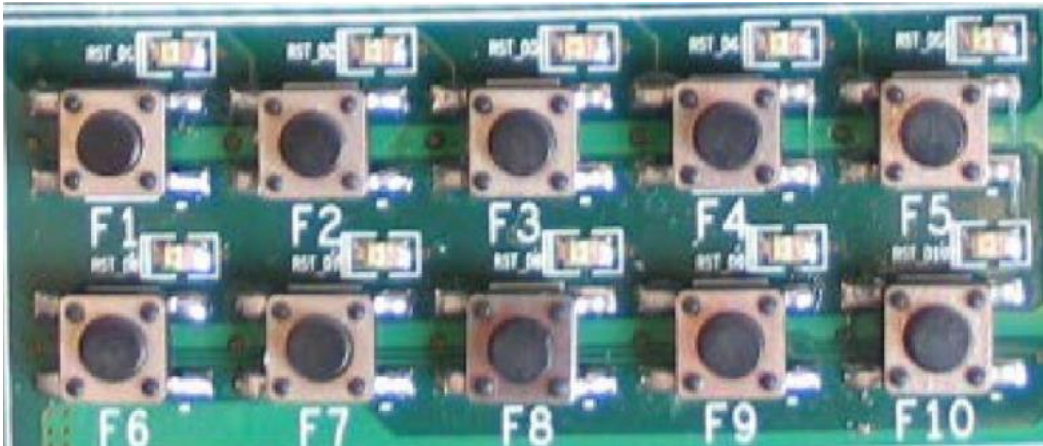
三、实验内容：

- 1. 在 FPGA 上实现无人机避障游戏
- 2. 显示设备： 16×16 LED 点阵
- 3. 特点：刷新扫描、像素合成、按键控制

四、硬件资源

设备仪器：FPGA 芯片（EP4CE10F17C8N）

【外设：点动开关】

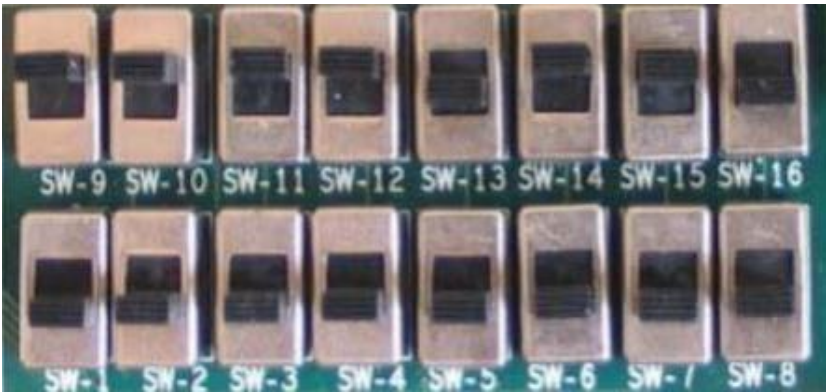


KSI 拨下时 F7-F10 可用

电动开关未按下时输出高电平，按下时输出低电平。

F1			CON1. 10	AB14
F2			CON1. 11	W20
F3			CON1. 12	AA15
F4			CON1. 13	AB15
F5			CON1. 14	T15
F6			CON1. 15	U20
F7	SMBUS_SDA	由开关 KSI 选择	CON1. 16	AA18
F8	SMBUS_SCL	由开关 KSI 选择	CON1. 17	AA16
F9	I2C_SCL	由开关 KSI 选择	CON1. 18	AB18
F10	I2C_SDA	由开关 KSI 选择	CON1. 19	AB19

【外设：拨码开关】

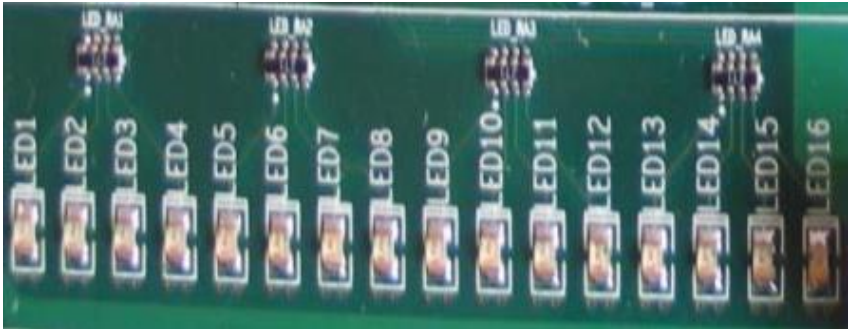


开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。

第一组：SW1，SW2，SW3，SW4，SW5，SW6，SW7，SW8 已经固定连接到实验平台中的 FPGA_CON1 和 FPGA_CON2 处；第二组：SW9，SW10，SW11，SW12，SW13，SW14，SW15，SW16 如果使用该模块，请 把控制拨码开关模块 CTRL_SW 中开关 SEL1，SEL2 拨置于下逻辑电平为 00，使 DP9 数码管显示 1。

M16	SW1
R19	SW2
AA14	SW3
AA1	SW4
A3	SW5
B4	SW6
C4	SW7
E6	SW8
V16	SW9
U14	SW10
B3	SW11
E5	SW12
A4	SW13
B5	SW14
A5	SW15
A6	SW16

【外设：LED 指示灯】



开发平台上有 2×8 共 16 个 LED 指示灯，便于对用户的操作给出提示，使用户得到相应的反馈信息，也可验证设计的正确性，低电平时亮。

第一组：LED1，LED2，LED3，LED4，LED5，LED6，LED7，LED8。如果使用该模块，请把控制拨码开关模块 LCD_ALONE_CTRL_SW 中开关 VLPO 拨置于下为低电平来确定。

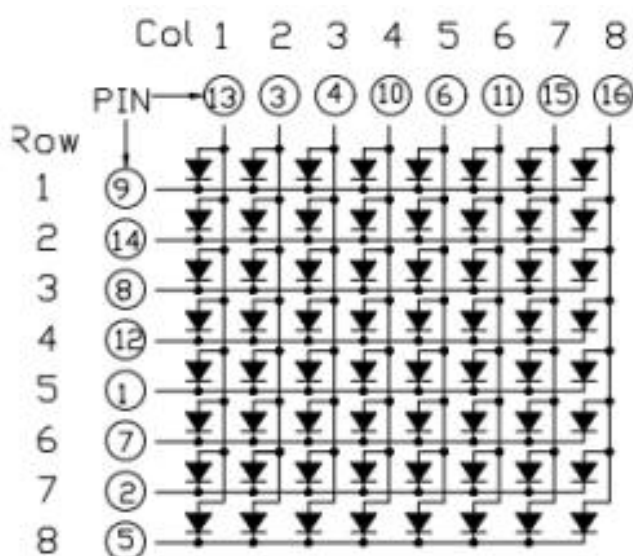
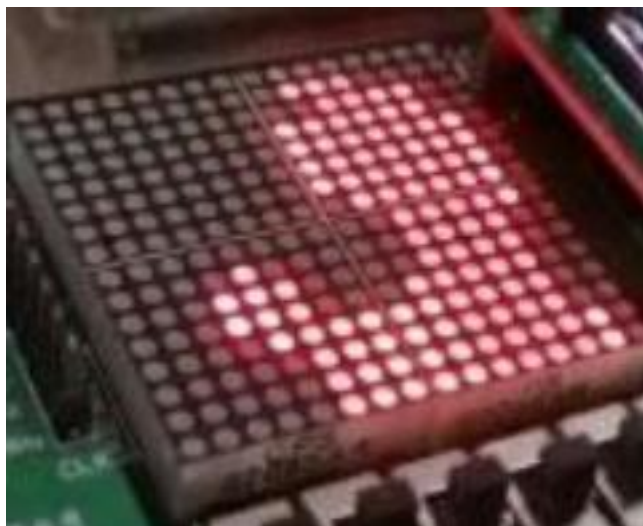
第二组：LED9，LED10，LED11，LED12，LED13，LED14，LED15，LED16。如果使用该模块，请把控制拨码开关 CTRL1_SW 中 SEL1 拨置于上，SEL2 拨置于下，逻辑电平为 10，使 DP9 数码管显示 2。

LED1	OV_D5	由开关 VLPO 选择	CON1.2	C10
LED2	OV_D6	由开关 VLPO 选择	CON1.3	E9
LED3	OV_D7	由开关 VLPO 选择	CON1.4	D10
LED4	OV_D8	由开关 VLPO 选择	CON1.5	V13
LED5	OV_D9	由开关 VLPO 选择	CON1.6	V14
LED6	VSNC	由开关 VLPO 选择	CON1.7	Y17
LED7	PCLK	由开关 VLPO 选择	CON1.8	W17
LED8	OV_RES	由开关 VLPO 选择	CON1.9	W19

【外设：8*8 点阵】

8*8 点阵扫描控制电路的光点是从显示模块左上角像素点为起始点扫描，终止于右下角像素点。本实验系统所用的点阵是行共阴、列共阳。

当列共阳为高电平，行共阴为低电平，则所接的发光像素点点亮，反之则所接的发光像素处于截止状态不发光。16*16 LED 点阵由 4 个 8*8 的 LED 阵级联而成。



点阵屏驱动：16 个行选，4 个列选。行选不一定为独热码，列选为 BCD 编码，建议一次更新一列。需合理地选择 ROM 的位宽以与扫描方式适配。

rom 位宽为 16，可以将这 16 位显示在一列上，每一位对应一个 led 灯。和动态扫描类似，只有一列是亮的，col[3:0]=3'd3 则表示第 3 列亮，即 rom 读出的数据显示在第 3 列。

row[4]	D19	B13	ROW12
row[3]	E19	B14	ROW13
row[2]	F18	A16	ROW14
row[1]	F20	B16	ROW15
row[0]	G20	B17	ROW16

五、实验过程与结果

第一部分：debouncing 模块（内容同之前两次实验，不过多赘述）

本模块一共有两个 always 模块，其中一个负责计时，另一个负责控制 key 信号的稳定。

每当时钟 clk_sys 的下降沿触发时，计数器 count 增加。如果 count 达到 delayT，并且当前按键未被按下（~key_b），且 key 是高电平（表示消抖完成），则将 count 复位为 0。

当计数器 count 达到 delayT-1，并且按键被按下时（~key_b），并且标志位 already_up 为高（表示按键之前已经是松开的），则将 key 设置为 0（表示按键已经按下），并且更新 already_up 为 0。

如果 key_b 为高（按键松开），则 already_up 被置为 1，表示准备检测下一次按下。如果 key 为低（按键已经按下），则将其重新设置为高（表示按键松开）。

第二部分：top 模块

```
1  module top(  
2      input wire clk_sys,  
3      input wire rstn_key,  
4      input wire flapn_key,  
5      input wire difficulty_mode,  
6      input wire cheat_mode,  
7      output wire [15:0] col_data,  
8      output wire [3:0] col_sel  
9  );  
10     wire rstn, flapn;  
11     // clocks  
12     wire refresh_clk;  
13     wire game_clk;  
14  
15     wire pause, rst_manual;  
16     debouncing debouncing_obj1(.clk_sys(clk_sys), .key_b(rstn_key), .key(rstn), .key_pulse());  
17     debouncing debouncing_obj2(.clk_sys(clk_sys), .key_b(flapn_key), .key(flapn), .key_pulse());  
18  
19     clock_divider #(  
20         .CLK_FREQ(25000000),  
21         .REFRESH_HZ(1200),  
22         .GAME_HZ(25)  
23     ) clkdiv (  
24         .clk_in(clk_sys),  
25         .rst(~rstn),  
26         .refresh_clk(refresh_clk),  
27         .game_clk(game_clk)  
28     );  
29     wire [255:0] frame_flat;  
30     wire game_over;  
31     game_logic game (  
32         .clk(game_clk),  
33         .rst(~rstn),  
34         .flap(~flapn),  
35         .difficulty_mode(difficulty_mode),  
36         .cheat_mode(cheat_mode),  
37         .frame_flat(frame_flat),  
38         .game_over(game_over)  
39     );  
40     led_driver driver (  
41         .clk(refresh_clk),  
42         .rst(~rstn),  
43         .frame(frame_flat),  
44         .col_data(col_data),  
45         .col_sel(col_sel)  
46     );  
47  
48     endmodule  
--
```

第三部分：clock_driver 模块

```
module clock_divider #(
    parameter integer CLK_FREQ = 25000000,
    parameter integer REFRESH_HZ = 1200,
    parameter integer GAME_HZ = 25
)(
    input wire clk_in,
    input wire rst,
    output reg refresh_clk,
    output reg game_clk
);

localparam integer REF_CNT_MAX = CLK_FREQ / (2*REFRESH_HZ);
localparam integer GAME_CNT_MAX = CLK_FREQ / (2*GAME_HZ);
//除以2是因为 $T_{out} = 2 * REF\_CNT\_MAX * T_{in}$ 
integer ref_cnt = 0;
integer game_cnt = 0;

always @(posedge clk_in or posedge rst) begin
    if (rst) begin
        ref_cnt <= 0;
        refresh_clk <= 0;
        game_cnt <= 0;
        game_clk <= 0;
    end else begin
        if (ref_cnt >= REF_CNT_MAX-1) begin
            ref_cnt <= 0;
            refresh_clk <= ~refresh_clk;
        end else ref_cnt <= ref_cnt + 1;

        if (game_cnt >= GAME_CNT_MAX-1) begin
            game_cnt <= 0;
            game_clk <= ~game_clk;
        end else game_cnt <= game_cnt + 1;
        //两个if-else是并行的
    end
end
endmodule
```

这里 $T_{out} = 2 * REF_CNT_MAX * T_{in}$,
其中 $T_{out} = 2\pi / REFRESH_HZ$
 T_{in} 是 $2\pi / CLK_FREQ$

第四部分：led_driver 模块

```
module led_driver(//这里在具体调用时候用的是refresh_clk
    input wire clk,
    input wire rst,
    input wire [255:0] frame,//16*16的屏幕
    output reg [15:0] col_data,
    output reg [3:0] col_sel
);
    reg [3:0] col_idx;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            col_idx <= 0;
            col_data <= 16'h0000;
            col_sel <= 4'd0;
        end else begin
            col_idx <= col_idx + 1;
            col_sel <= col_idx;
            col_data <= frame[col_idx*16+:16];
        end
        // 当 col_idx = 15 时 (最后一列):
        // 起始位 = 15 * 16 = 240。
        // 代码变为 frame[240+:16]。
        // 这会选择 frame 从第240位开始, 总共16位, 也就是 frame[255:240]。
        // 传统写法: col_data <= frame[ (col_idx * 16) + 15 : (col_idx * 16)];
    end
endmodule
```

当 col_idx = 15 时 (最后一列):

起始位 = 15 * 16 = 240。

代码变为 frame[240+:16]。

这会选择 frame 从第 240 位开始, 总共 16 位, 也就是 frame[255:240]。

传统写法: col_data <= frame[(col_idx * 16) + 15 : (col_idx * 16)];

第五部分：Lfsr8 模块

```
module lfsr8(
    input wire clk,
    input wire rst,
    input wire advance,
    output reg [7:0] out
);
    reg [7:0] state;
    wire feedback = state[7] ^ state[5] ^ state[4] ^ state[3];

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state <= 8'hA5; // seed设置为一个预定义的初始值
            //种子为随机序列提供了一个确定的起点。种子不能是0
            //因为如果 state 是全0, 那么 feedback 的计算结果 (0^0^0^0) 也永远是0, LFSR就会被“卡”在0状态。
            out <= 8'hA5;
        end else begin
            if (advance) begin
                state <= {state[6:0], feedback};
                out <= {state[6:0], feedback}; //移位寄存器, advance=1时前进, acdvanc=0时锁存
            end
        end
    end
endmodule
```

核心模块：game_logic

```
module game_logic(
    input wire clk,
    input wire rst,
    input wire flap,
    input wire difficulty_mode,
    input wire cheat_mode,
    output [255:0] frame_flat,
    output reg game_over
);

localparam integer WIDTH = 16;
localparam integer HEIGHT = 16;
localparam integer BIRD_X = 3;
localparam integer GAP_H = 4; //把空隙加宽
localparam integer DECISION_COLUMN = BIRD_X + 3;

localparam REFRESH_PERIOD_EASY = 15; // 简单模式：每16拍更新一次 (0-15)
localparam REFRESH_PERIOD_HARD = 7; // 困难模式：每8拍更新一次 (0-7)，速度翻倍

reg [15:0] frame [0:15]; //顺序!!
assign frame_flat = {frame[15], frame[14], frame[13], frame[12],
    frame[11], frame[10], frame[9], frame[8],
    frame[7], frame[6], frame[5], frame[4],
    frame[3], frame[2], frame[1], frame[0]};

reg [15:0] playfield [0:15]; //只有管道，frame是最终的画面
reg [3:0] bird_y;

wire [7:0] rnd_out;
reg rnd_advance;
lfsr8 rnd(.clk(clk), .rst(rst), .advance(rnd_advance), .out(rnd_out));

localparam [2:0] BIRD_P0 = 3'b010;
localparam [2:0] BIRD_P1 = 3'b111;
localparam [2:0] BIRD_P2 = 3'b010; //形状

integer i, j; // 循环迭代器

reg [15:0] bird_col0, bird_col1, bird_col2;
reg [3:0] gap_y;
reg [15:0] gap_mask;
reg [15:0] new_pipe_col;
reg [3:0] map_refresh_cnt;

//优化1
reg [2:0] gap_counter;
//优化2
wire [3:0] current_refresh_period;
// 使用三元运算符进行选择：(条件) ? (真值) : (假值)
assign current_refresh_period = difficulty_mode ? REFRESH_PERIOD_HARD : REFRESH_PERIOD_EASY;
//把核心代码里：管道移动、小鸟下落判断等于1111都改为current_refresh_period

//优化3
reg pipe_in_sight; // 标志位：前方视野内是否有管道
reg [3:0] next_pipe_gap_y; // 下一个管道缺口的Y坐标 (底部)
reg [3:0] target_y; // 作弊模式目标Y坐标
```

这里展示了相比于原始代码本人所做的三点优化（变量定义）

优化一部分代码：

```
if (map_refresh_cnt == current_refresh_period) begin
    // 管道向左移动一列
    for (i = 0; i < WIDTH-1; i = i + 1) begin
        playfield[i] <= playfield[i+1];
    end

    rnd_advance <= 1;
    gap_y = rnd_out[3:0] % (HEIGHT - GAP_H + 1);
    gap_mask = ((16'hFFFF >> (16 - GAP_H)) << gap_y);
    new_pipe_col = ~gap_mask;
    // 检查间隔计数器是否小于5
    if (gap_counter < 5) begin
        // 如果间隔不够，必须强制插入一个空白列
        playfield[15] <= 16'h0000;
        // 间隔计数器加1
        gap_counter <= gap_counter + 1;
    end else begin // 如果间隔已经足够（大于等于5）
        if (rnd_out[4]) begin //
            playfield[15] <= new_pipe_col;
            // 重要：生成管道后，必须将间隔计数器清零！
            gap_counter <= 3'd0;
        end else begin // 随机决定继续留空
            playfield[15] <= 16'h0000;
            gap_counter <= gap_counter + 1;
        end
    end
end
end
```

优化二部分代码：

```
if (!game_over) begin
    // map_refresh_cnt<=map_refresh_cnt+1; //原始代码
    map_refresh_cnt <= (map_refresh_cnt == current_refresh_period) ? 4'd0 : map_refresh_cnt + 1;
    //因为现在刷新周期不固定，所以这里也要改
    //意思是当计数器达到当前刷新周期时，重置为0，否则加1
    if (cheat_mode) begin
        if (bird_y < target_y && bird_y < HEIGHT - 2) bird_y <= bird_y + 1;
        else if (bird_y > target_y && bird_y > 0) bird_y <= bird_y - 1;
    end else begin
        if (flap) begin
            if (bird_y < HEIGHT - 2) bird_y <= bird_y + 1;
            else bird_y <= HEIGHT - 2;
        end else begin
            if (map_refresh_cnt == current_refresh_period) begin
                if (bird_y > 0) bird_y <= bird_y - 1;
                else bird_y <= 0;
            end
            else bird_y <= bird_y;
        end
    end
end
```

优化三最重要的感知逻辑:

// 作弊模式的“感知”逻辑

```
always @(*) begin
```

```
reg found_gap;
```

```
pipe in sight = 1'b0;
```

```
next pipe gap y = 4'd7; // 默认目标
```

```
target_y = bird_y; //这里改为保持当前位置
```

```
found_gap = 1'b0; // 默认没找到
```

```
if (playfield[DECISION_COLUMN] != 16'h0000) begin
```

```
pipe_in_sight = 1'b1;
```

```
// 这个 for 循环现在边界是固定的 (0 to 15)
```

```
for (j = 0; j < HEIGHT; j = j + 1) begin
```

// 只有在【还未找到缺口】的情况下，才开始找

```
if (playfield[DECISION_COLUMN][j] == 1'b0 && !found_gap) begin
```

```
next_pipe_gap_y = j; // 记录第一个找到的缺口底部
```

```
found_gap = 1'b1; // 标记已经找到缺口
```

end

end

end

```
if (cheat mode) begin
```

```
if (pipe_in_sight) begin
```

```
target y = next pipe gap y + (GAP H / 2) - 1;
```

```
end else begin
```

```
target_y = bird_y; // 保持当前位置
```

end

end

end

位置更新部分已经在优化二的图片中显示，

```
if (cheat_mode) begin
```

```
if (bird_y < target_y && bird_y < HEIGHT - 2) bird_y <= bird_y + 1;
```

```
else if (bird y > target y && bird y > 0) bird y <= bird y - 1;
```

end else begin

管脚绑定情况界面：**【此处新引入了两处 input 所以新定义了两个拨码开关】**

[illegible]

【优化总结】：

1. 相邻管道之间至少间隔 5 列以上，确保地图有足够的距离供玩家调整
2. 设置两种游戏模式（困难和简易），由拨码开关控制，两种模式的区别主要体现在地图和无人机（小鸟）的刷新/下降速度不同
3. Cheat 模式，实现自动巡航，感知前方两列的路障空隙位置，并找到空隙的最低位置，将更新目标位置设定在空隙中心，按照下面代码实现移动

```
if (bird_y < target_y && bird_y < HEIGHT - 2) bird_y <= bird_y + 1;
```

```
else if (bird_y > target_y && bird_y > 0) bird_y <= bird_y - 1;
```

可以看到无人机/小鸟从原始位置逐渐移动，而不是瞬移

【如果地图更新过快，如困难模式，且相邻两列的空隙 y 方向上差距过大，则仍然会在移动过程中碰撞】

最终结果：

详细见文件夹内视频

三部分优化均有展示

六、实验仿真波形

【省略 top.v 仿真波形，于 16*16 点阵上观测调试】

仅保留 debouncing 的仿真部分

编写 debouncing_tb.v 代码

```
`timescale 1ns/1ps

module debouncing_tb;

    // -----
    // 时钟产生 (周期10ns = 100MHz)
    // -----
    reg clk_sys = 0;
    always #5 clk_sys = ~clk_sys;

    // -----
    // DUT 输入 / 输出
    // -----
    reg key_b;
    reg resetn;
    wire key;
    wire key_pulse;
    wire [7:0]counter;

    // -----
    // DUT 实例化 (delayT很小方便观察)
    // -----
    debouncing_fsm #(
        .bitwidth(8),
        .delayT(50)//这里修改了位宽和delayT
    ) uut (
        .clk_sys(clk_sys),
        .resetn(resetn),
        .key_b(key_b),
        .key(key),
        .key_pulse(key_pulse),
        .counter(counter)
    );

    // 波形输出
    initial begin
        $dumpfile("debouncing.vcd");
        $dumpvars(0, debouncing_tb);
    end

    // 测试流程
    initial begin
        $display("=== Debouncing Testbench Start ===");
        // 初始状态
        resetn = 1;
        #50;
        // 复位
        resetn = 0;
        #30;
        resetn = 1;
        #50;

        key_b = 1; // 初始未按下
        #50;

        // 模拟有抖动的按键按下
        // 抖动序列: 1->0->1->0->1->0->保持0
        key_b = 0; #2;
        key_b = 1; #2;
        key_b = 0; #2;
        key_b = 1; #2;
        key_b = 0; #100; // 稳定按下 (低电平)

        // 保持按下状态一段时间
        #100;

        // 模拟抖动释放: 0->1->0->1->保持1
        key_b = 1; #2;
        key_b = 0; #2;
        key_b = 1; #2;
        key_b = 0; #2;
        key_b = 1; #100;

        // 等待波形稳定
        #200;

        $display("=== Simulation Finished ===");
        $finish;
    end
end
```

1) 编译文件

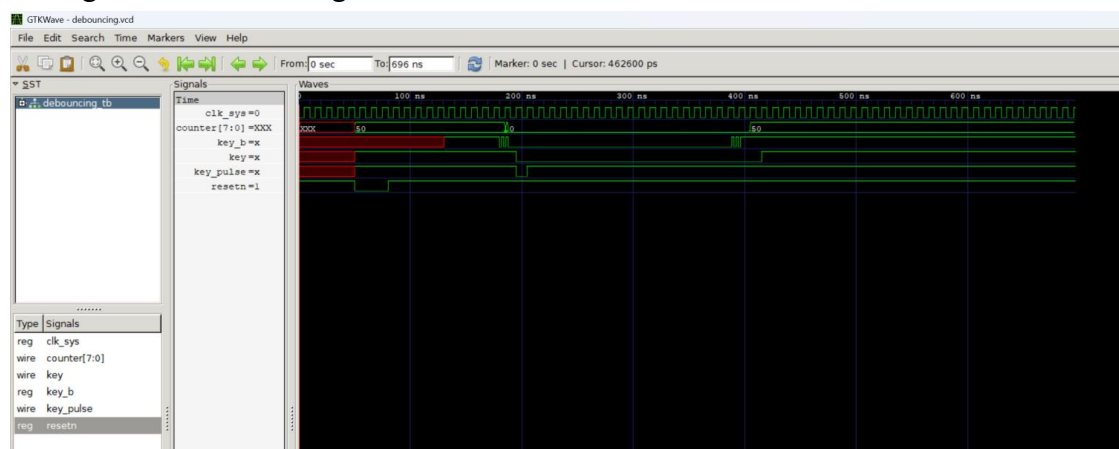
命令行: iverilog -o debouncing.out debouncing.v debouncing_tb.v

2) 生成 .vcd 波形文件

命令行: vvp debouncing.out

3) 观察波形

命令行: gtkwave debouncing.vcd



七、附加实验仿真波形

无附加实验，两个实验均为必做实验。